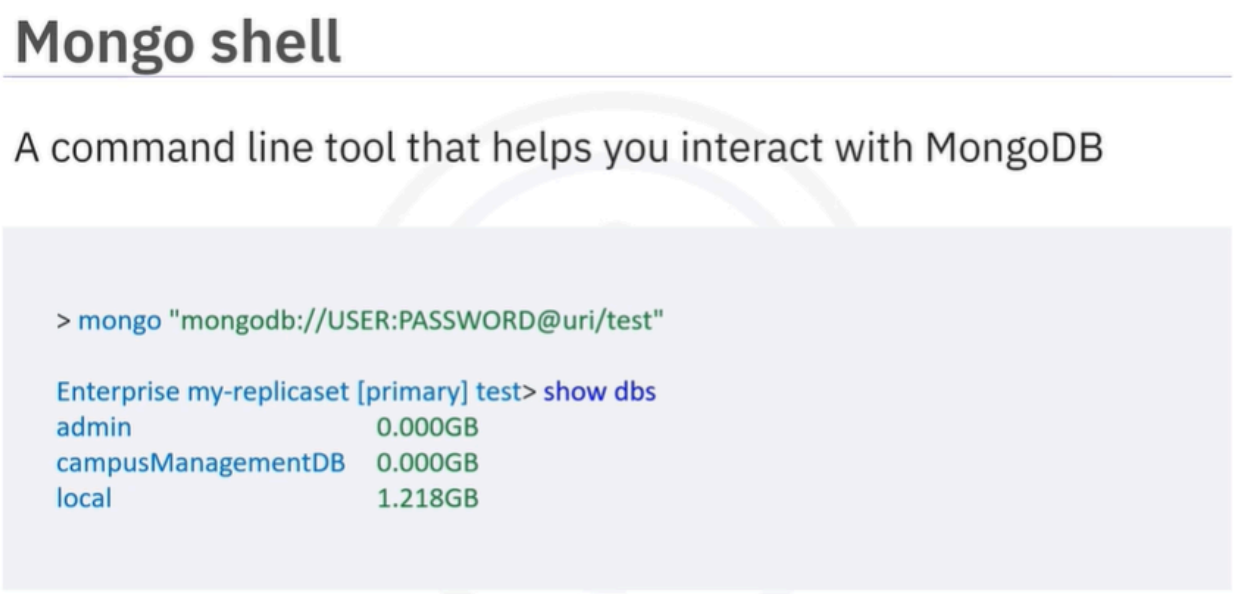
CRUD Operations



- MongoShell is a command-line tool provided by MongoDB to interact with your databases. It is an interactive JavaScript interface and you can use it to perform data and administrative operations on MongoDB. So, first we connect to our cluster by providing a connection string. And once connected, we can view the list on all our databases.

Mongo shell

Set database context

```
> use campusManagementDB
switched to db campusManagementDB
> show collections
staff
students
```

- For simplicity, we will assume the shell is already connected to our MongoDB instance. To start working on the Campus Management database, we will run the following statement. And then we can see what collections are present in the Campus Management database using this command. It shows two collections, staff and students.

Create

```
> db.students.insertOne({
                        "firstName": "John",
                        "lastName": "Doe",
                        "email": "john.doe@email.com",
                        "studentId": 20217484
                    })
{
        "acknowledged" : true,
        "insertedId" : ObjectId("60424dad68ad40a3490e0db4")
}
```

- CRUD operations are create, read, update and delete operations, hence the acronym CRUD. Let's start with a create operation. We will insert a new document into the students collection. It takes the form of db, which is the Campus Management db database, then the collection name, which is students, and then the insert1 function, which takes an argument as the document we want to create in the students collection.
- The outcome is an acknowledgment that the operation was successful. And since this was an insert operation, it shows inserted ID. Underscore ID is a required field on every document in MongoDB. Because we didn't provide one ourselves in John Doe's document, the MongoShell driver will add the underscore ID field and generate the object ID for us.

Create Many

```
> students_list = [
{ "firstName": "Sarah", "lastName": "Jane", "email": "sarah@mail.com", "studentId": 20215124 },
{ "firstName": "Peter", "lastName": "Parker", "email": "peter@web.com", "studentId": 20218873 }]

> db.students.insertMany(students_list)
{
      "acknowledged" : true,
            "insertedIds" : [
                  ObjectId("60424ee468ad40a3490e0db5"),
                  ObjectId("60424ee468ad40a3490e0db6")
                        ]
}
```

- MongoShell is a JavaScript interpreter, meaning you can define variables and perform other functions in it too. Here, students underscore list is created with two documents in it, and we can pass this as an argument to the insertMany function.

Read

```
> db.students.findOne()
{
        "_id" : ObjectId("60421b5c26fe73f38fdd83db"),
        "firstName" : "John",
        "lastName" : "Doe",
        "email" : "johnd@campus.edu",
        "studentId" : 20217484
}
```

- Now let's do some read operations. In the first one, we call the find1 function, without any filter criteria. This will return the first document in natural order, which is the order in which the database refers to documents on disk.

Read

```
> db.students.findOne({ "email": "sarah@mail.com" })
{
        "_id" : ObjectId("604228bee45df18c77dc2dd7"),
        "firstName" : "Sarah",
        "lastName" : "Jane",
        "email" : "sarah@mail.com",
        "studentId" : 20215124
```

Read

```
> db.students.findOne({ "email": "sarah@mail.com" })
{
    "_id" : ObjectId("604228bee45df18c77dc2dd7"),
    "firstName" : "Sarah",
    "lastName" : "Jane",
    "email" : "sarah@mail.com",
    "studentId" : 20215124
}
```

Read

```
> students.find({ "lastName": "Doe" })
{ "_id" : ObjectId("60421b5c26fe73f38fdd83db"), "firstName" : "John", "lastName" : "Doe",
"email" : "john.doe@email.com", "studentId" : 20217484 }
{ "_id" : ObjectId("60421b5c26fe73f38fdd84ab"), "firstName" : "Mark", "lastName" : "Doe",
"email" : "mark@mail.com", "studentId" : 20215001 }
```

Read

```
> students.count({ "lastName": "Doe" })
2
```

- In this second read operation, we want to find the first student with a specific email address. If more than one document matches the criteria, only the first one will be read, as we are using the find1 function. Next, we want to retrieve all students with the last name of Doe. And finally, if we want to count how many students have the last name Doe, we call the count function. This count will happen on MongoDB, and only the number value will be returned.

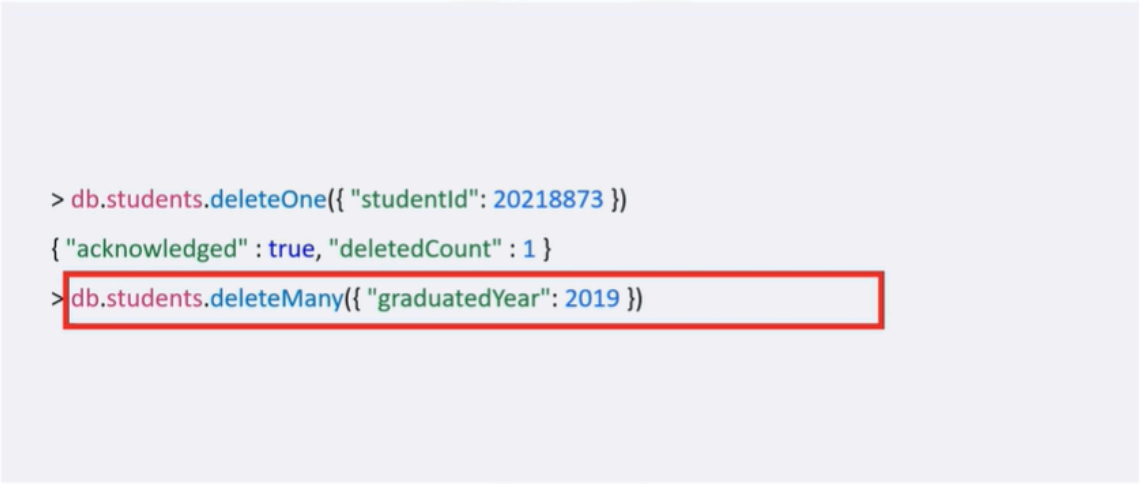
Replace

```
> student = db.students.findOne({ "lastName": "Doe" })
> student["onlineOnly"] = true
> student["email"] = "johnd@campus.edu"
> db.students.replaceOne({ "lastName": "Doe" }, student)
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
```

- Now we will perform a replace operation. Let's retrieve a student record and make some changes to the document. We are creating a new field to identify which students are on campus or online only. We will also update the email with campus email.
- Once all changes are made, we will call the replace1 function, which takes as its first argument the filter criteria to find a document with the last name of Doe. The second argument is the updated student object. The outcomes of this statement are an acknowledgment, the information about how many documents matched our filter criteria, and the count of how many were actually modified.

- In the replace example, we were sending the whole document back with the new changes. For larger documents, this will take a lot of transfer time between the client and the database. Instead, we can use MongoDB's in-place updates for small changes. So, we do this by constructing a changes document, which in our case only has two assignments, which come under $set. Then we would call the update1 function.
- To update all of the students to online only due to lockdown, we can run the updateMany function without any filter criteria and a change document.

Delete



- To run a delete operation like update and find, the delete1 function takes filter criteria as a first argument. Running that delete statement provides you with an acknowledgment and a number representing how many documents were deleted.
- Similarly, you have a deleteMany function to delete more than one document meeting the criteria.

Recap

In this video, you learned that:

- The Mongo shell is an interactive command line tool provided by MongoDB to interact with your databases
- To use the Mongo shell, you first need to make a connection to your cluster via a connection string
- You use ‘show dbs’ to list databases, ‘use *nameofdatabase*’ to select a database, and ‘show collections’ to list collections in a database
- CRUD operations consist of Create, Read, Update, and Delete
- Useful functions include insertOne, insertMany, findOne, find, count, replace, updateOne, updateMany, deleteOne, and deleteMany
- Different kinds of acknowledgments are returned based on the operation run